

INTERNAL WORKSHOP · 2 HOURS

Kotlin Multiplatform

Write the logic once. Run it natively on Android, iOS and desktop.

Tomer Glick

THE PROBLEM

One feature, written three times

Android

Kotlin: Retrofit, Room and its own copy of the validation rules.

iOS

Swift: URLSession, Core Data and the same rules written again.

Web or desktop

TypeScript or Java: yet another client and the rules once more.

Same business rules, three implementations, **three sets of bugs** that drift apart over time.

Share what should be the same. Keep native what should be native.

Kotlin Multiplatform lets you write code once in Kotlin and use it on Android, iOS, desktop, web and server. Shared code compiles to each platform's own native format: no JavaScript bridge, and no virtual machine on iOS.

FLEXIBLE BY DESIGN

You choose how much to share

Less shared

More shared

One module

Share a single piece first, like validation rules or an API client. Your apps keep everything else.

All the logic

Networking, storage, rules and ViewModels are shared. Each platform keeps its native UI: Jetpack Compose on Android, SwiftUI on iOS.

What we build today

Logic and UI

Compose Multiplatform shares the screens as well, so almost the whole app lives in common code.

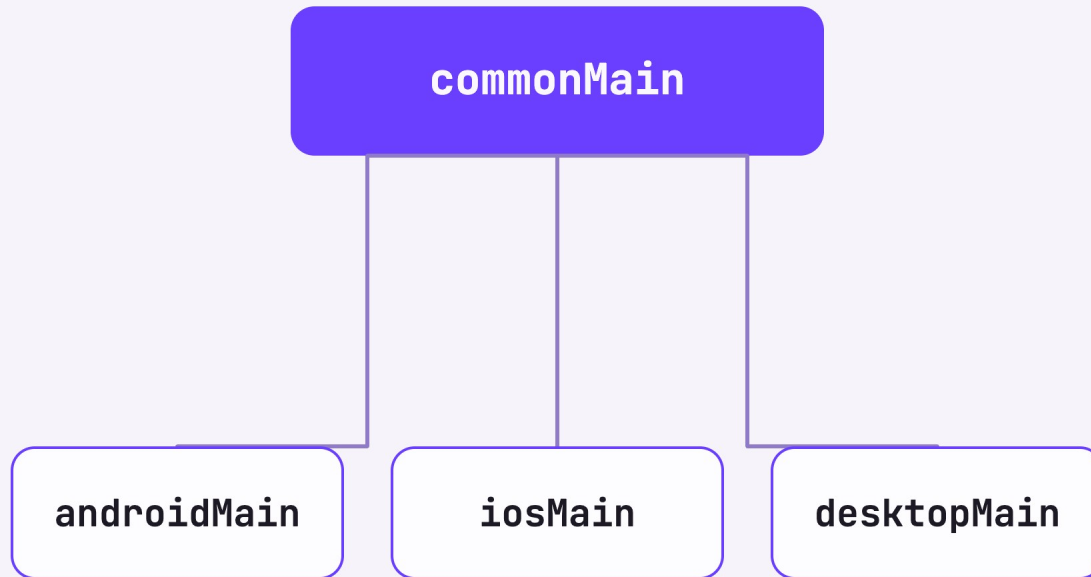
HOW IT WORKS

Compiled natively for each platform



The Kotlin compiler has a backend for each platform. Your common code goes through the right one for every target you declare.

Source sets decide where code lives



Platform code sees `commonMain`, never the other way around

`commonMain`

Pure Kotlin and multiplatform libraries. No `java.*`, no `android.*`, no `UIKit`.

`androidMain`, `iosMain`...

Full access to each platform's APIs. Entry points and actual implementations live here.

`commonTest`

Tests written once, run on every target.

PLATFORM-SPECIFIC CODE

expect / actual

```
commonMain / Platform.kt  
expect fun platformName(): String
```

```
androidMain  
actual fun platformName() =  
    "Android " +  
    Build.VERSION.RELEASE
```

```
iosMain  
actual fun platformName() =  
    UIDevice.currentDevice  
        .systemName()
```

```
desktopMain  
actual fun platformName() =  
    System.getProperty(  
        "os.name")
```

The compiler checks that every target has an actual. For bigger platform services, a plain interface with one implementation per platform is often simpler to test.

ECOSYSTEM

The libraries you'll reach for

Need	Library	In our project
Networking	Ktor client	Today
JSON	kotlinx.serialization	Today
Async code	kotlinx.coroutines	Today
ViewModel	androidx.lifecycle	Today
Local database	SQLDelight or Room	Next steps
Key-value storage	multiplatform-settings or DataStore	Next steps
Dependency injection	Koin	Next steps

SHARED UI

Compose Multiplatform

JetBrains' UI framework, built on Jetpack Compose. Same @Composable functions and the same mental model as modern Android, on every platform.

```
@Composable
fun Greeting() {
    Text("Hello from ${platformName()}")
}
```

Platform support

Android

Stable

iOS

Stable

Desktop

Stable

Web

Beta

iOS support became stable in May 2025, with version 1.8.

CHOOSING A TOOL

How KMP compares

	Kotlin Multiplatform	Flutter	React Native
Language	Kotlin	Dart	JavaScript or TypeScript
UI	Native UI, or shared with Compose Multiplatform	Flutter's own widgets, drawn by its engine	React components mapped to native views
How much is shared	Anything from one module to the whole app	Usually the whole app	Usually the whole app
Platform APIs	Called directly from Kotlin code	Through platform channels or FFI	Through native modules

The key difference: KMP doesn't ask you to replace your UI or rewrite your app. You can adopt it **one module at a time**.

MATURITY

Stable and production-ready

2023

Declared stable by JetBrains in November 2023

2024

Officially supported by Google, announced at Google I/O

Used in production at companies like Netflix, McDonald's, Cash App and Forbes, and in Google Docs on iOS.

TRADE-OFFS

What to know before you commit



iOS builds need a Mac

Xcode is required to build and sign iOS apps.
Android and desktop work on any machine.



Swift sees Kotlin via Objective-C

Sealed classes and suspend functions feel clunky in Swift. SKIE helps; direct Swift export is on the way.



Slower iOS builds

Kotlin/Native builds take longer than JVM builds,
especially the first clean build.



The whole team reads Kotlin

iOS developers need to review and debug shared Kotlin code, even if they rarely write it.

TODAY

The plan for the next two hours

0:00 What is KMP **You are here**

0:15 Tour the starter project

0:35 expect / actual

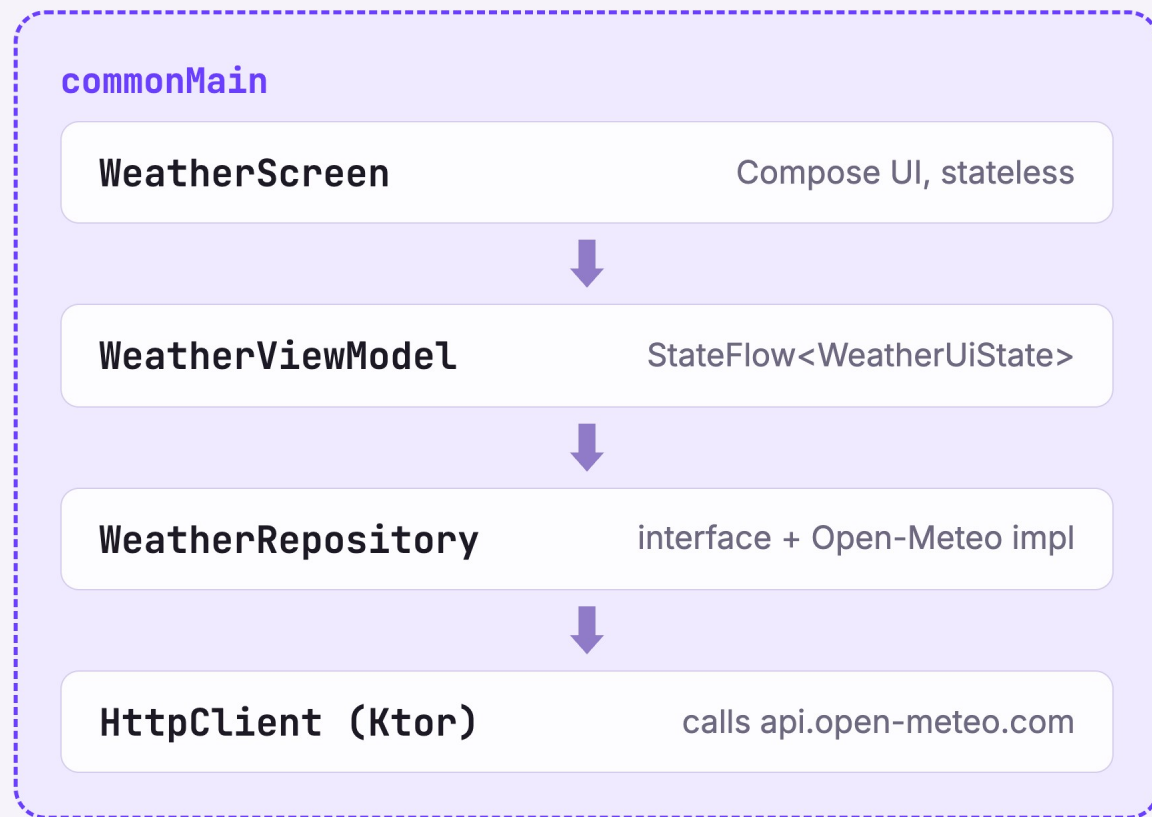
0:55 Break

1:05 Networking and a shared ViewModel

1:30 Final lab: Mini Weather

1:55 Wrap-up and next steps

What we're building: Mini Weather



Platform source sets

`androidMain` → `MainActivity`

`iosMain` → `MainViewController`

`desktopMain` → `main()`

Each also has one actual `platformName()`.

Everything in the dashed box is written once.

HANDS-ON TIME

Let's build it

```
git checkout step-0-starter
```

Then run `./gradlew :composeApp:run` and you should see "Hello, KMP!"

STEP 0 · 15 MINUTES

Tour the starter project

`settings.gradle.kts`

Repositories and the single module, `:composeApp`

`gradle/libs.versions.toml`

Every library and plugin version in one place

`composeApp/build.gradle.kts`

Targets, and the dependencies of each source set

`composeApp/src/*Main/`

One entry point per platform, calling the same `App()`

Run it

```
# Desktop
./gradlew :composeApp:run
```

```
# Android
Run config: composeApp
```

Try this

Change the text in `App.kt`, then run on both platforms. One edit, two apps.

The kotlin { } block

composeApp/build.gradle.kts (simplified)

```
kotlin {  
    androidTarget()  
    iosArm64(); iosSimulatorArm64()  
    jvm("desktop")  
  
    sourceSets {  
        commonMain.dependencies {  
            implementation(libs.ktor.client.core)  
        }  
        androidMain.dependencies {  
            implementation(libs.ktor.client.okhttp)  
        }  
    }  
}
```

Targets

The platforms Gradle compiles for. iOS targets are skipped on Windows and Linux.

Source sets

Each one gets its own dependencies. commonMain may only use multiplatform libraries.

Same API, per-platform engine

Ktor's core is shared; each platform adds its own HTTP engine.

STEP 1 · 20 MINUTES

Add your first expect / actual

1

Declare it

In commonMain, create Platform.kt with expect fun platformName(): String

2

Implement it

Add an actual in androidMain, desktopMain and iosMain, using Build.VERSION, System.getProperty and UIDevice.

3

Use it

Show the result in App.kt, then run on Android and desktop.



Try breaking it: delete one actual and build. The compiler won't let you forget a platform.

0:55 – 1:05

Break • 10 minutes

Behind? Run the command below and you're caught up for step 2.

```
git checkout step-1-expect-actual
```

STEP 2 · 25 MINUTES

Model the API response

data/WeatherModels.kt

```
@Serializable
data class ForecastResponse(
    val current: CurrentWeather,
)

@Serializable
data class CurrentWeather(
    val time: String,
    @SerializedName("temperature_2m")
    val temperature: Double,
)
```

data/HttpClientFactory.kt

```
fun createHttpClient() = HttpClient {
    install(ContentNegotiation) {
        json(Json {
            ignoreUnknownKeys = true
        })
    }
}
```

No engine is passed: Ktor picks the one each platform depends on (OkHttp, CIO or Darwin).

Fetch the data and expose state

data/WeatherRepository.kt

```
override suspend fun getCurrentWeather(
    latitude: Double, longitude: Double,
): CurrentWeather = client.get(
    "https://api.open-meteo.com/v1/forecast"
) {
    parameter("latitude", latitude)
    parameter("longitude", longitude)
    parameter("current", "temperature_2m,...")
}.body<ForecastResponse>().current
```

ui/WeatherViewModel.kt (simplified)

```
class WeatherViewModel(
    private val repository: WeatherRepository
) : ViewModel() {
    private val _weather =
        MutableStateFlow<CurrentWeather?>(null)
    val weather = _weather.asStateFlow()

    fun loadLisbon() {
        viewModelScope.launch {
            _weather.value = repository
                .getCurrentWeather(38.72, -9.14)
        }
    }
}
```

Observe the ViewModel from Compose

App.kt (simplified)

```
@Composable
fun App() {
    val viewModel = viewModel { WeatherViewModel() }
    val weather by viewModel.weather.collectAsState()

    Button(onClick = viewModel::loadLisbon) {
        Text("Load weather")
    }
    weather?.let { Text("${it.temperature} °C") }
}
```

viewModel { }

Outside Android there's no reflection, so we tell it how to create the ViewModel.

Desktop gotcha

Dispatchers.Main on desktop needs kotlinx-coroutines-swing. It's already in the build file.

Build Mini Weather

- 1 Add a list of cities and show it in a LazyColumn
- 2 Replace the nullable state with a sealed WeatherUiState
- 3 selectCity(): emit Loading, then Success or Error, with a Retry button
- 4 Show platformName() in a footer

Stretch goals and hints are in LAB.md.

ui/WeatherUiState.kt

```
sealed interface WeatherUiState {  
    data object Idle : WeatherUiState  
    data class Loading(val city: City) : WeatherUiState  
    data class Success(  
        val city: City, val weather: CurrentWeather,  
    ) : WeatherUiState  
    data class Error(  
        val city: City, val message: String,  
    ) : WeatherUiState  
}
```

WRAP-UP

What you learned today

Source sets:
shared code and
platform code

expect / actual for
small platform hooks

Shared networking
and ViewModels

One Compose UI on
every platform

Next steps

kotlinlang.org/docs/multiplatform

The official docs and tutorials

kmp.jetbrains.com

Generate new KMP projects

klibs.io

Search for multiplatform libraries

Questions?

Workshop repo: github.com/TomerGlick/KMP-Starter-Project

Feedback: [\[link to form\]](#)

BEFORE THE SESSION

Set up ahead of time

1 Install JDK 17 or newer

Or use the JDK bundled with Android Studio. Check it with `java -version`.

2 Install Android Studio

Latest stable version, then add the Kotlin Multiplatform plugin from Settings > Plugins. Set up an emulator too.

3 Clone and open the repo

Clone github.com/TomerGlick/KMP-Starter-Project, open it and wait for Gradle sync. The first sync can take 10–20 minutes.

4 Run it once

Desktop: `./gradlew :composeApp:run`. Android: the `composeApp` run config. You should see “Hello, KMP!”



Corporate network? Gradle must reach Maven Central, Google Maven and the Gradle Plugin Portal.